

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ
----ô&ò----

TÀI LIỆU THÍ NGHIỆM VI XỬ LÝ

GVHD: ThS. Nguyễn Đức Duy

Sinh viên thực hiện:

Số hiệu sinh viên:

Mã học phần:

Hà Nội, 2025

Kit thí nghiệm SN8F5708_EVK

1. Tổng quan về Kit

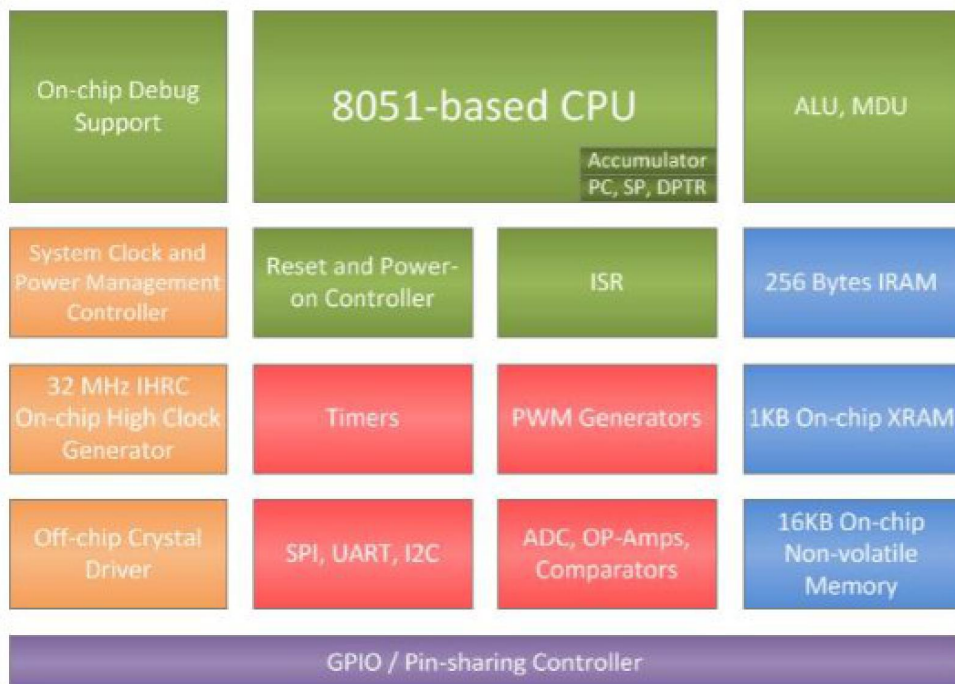
1.1 Giới thiệu chip SN8F5708

1.1.1 Các đặc trưng cơ bản

SN8F5708 là vi điều khiển kiến trúc 8-bit tương thích với bộ lệnh MCS-51, nói cách khác SN8F5708 là vi điều khiển 8051 nâng cao. Nó chạy nhanh hơn 8051 gốc 12,1 lần ở cùng tần số với các đặc trưng cơ bản sau:

- Tần số CPU linh hoạt lên đến 32 MHz
- 256-byte RAM nội bộ (IRAM)
- Bộ nhớ flash 16 KB (IROM)
- Hỗ trợ chương trình trong hệ thống
- 19 nguồn ngắt với điều khiển ưu tiên
- 3 ngắt ngoài: INT0, INT1, INT2
- 3 bộ hẹn giờ 8/16-bit với một lần chụp/so sánh
- 3 bộ tạo PWM 8/16-bit với điều khiển dải chết
- Bộ ADC SAR 12-bit với 12 kênh ngoài
- Giám sát có thể lập trình và thiết lập lại bên ngoài
- Giao diện SPI, UART, I2C với hỗ trợ SMBus
- Hỗ trợ gỡ lỗi trên chip (giao diện gỡ lỗi một dây)
- Điện áp cung cấp rộng (1,8 V – 5,5 V)
- Nhiệt độ làm việc rộng (-40 °C đến 85 °C)
- RAM ngoài 1 KB (XRAM)

1.1.2 Sơ đồ khối



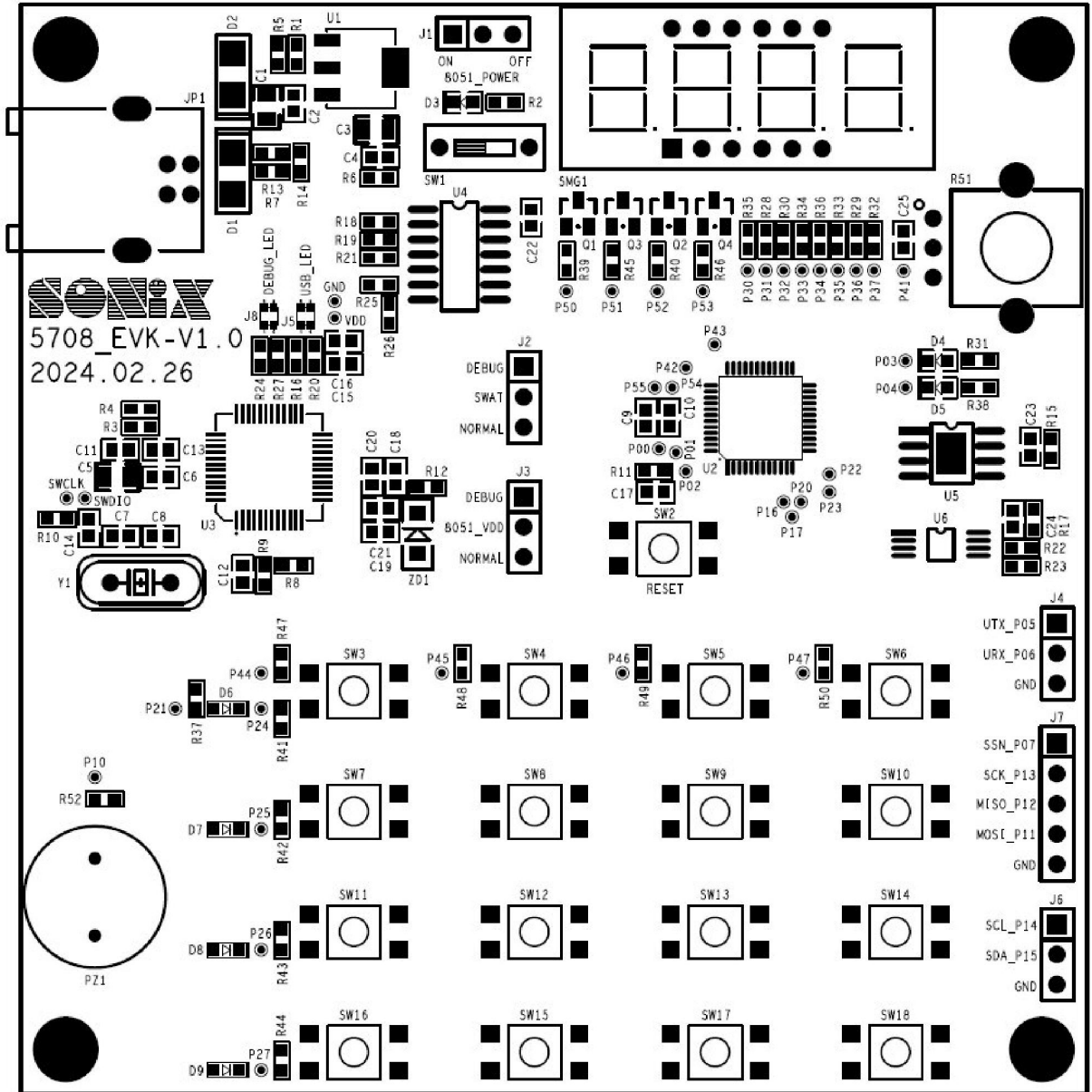
Hình 1. Sơ đồ khối vi điều khiển SN8F5708

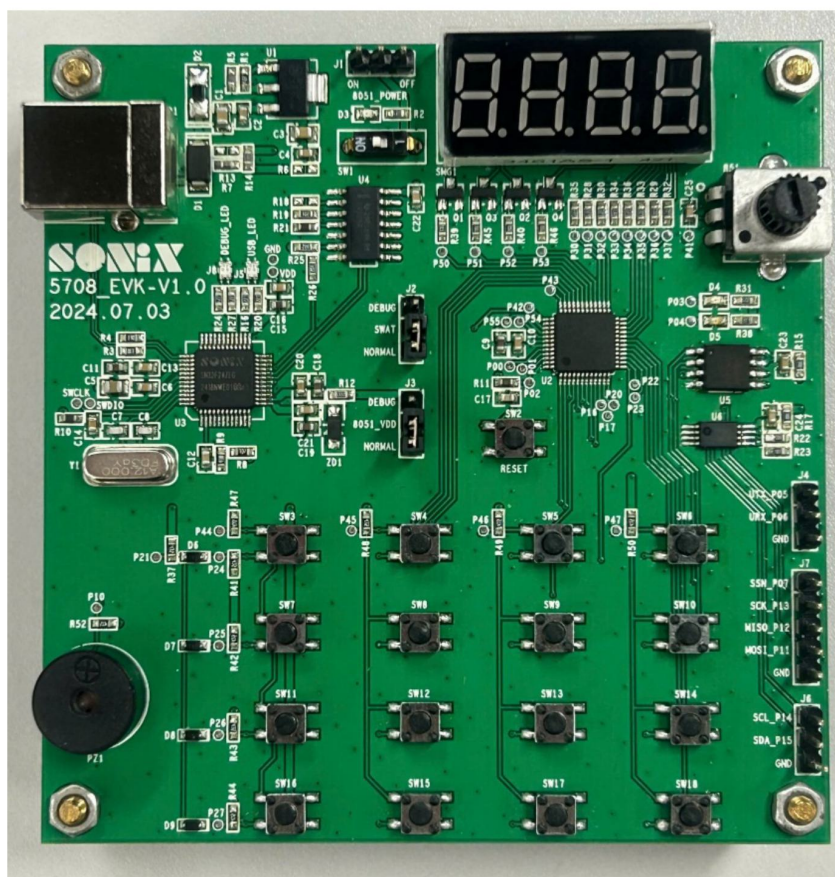
Bộ vi điều khiển SN8F5708 là một máy tính nhỏ, trong đó CPU, bộ nhớ (RAM, ROM), I/O thiết bị ngoại vi, timers, counters, ... được nhúng vào trong một mạch tích hợp. Các bộ vi xử lý và tất cả các khối này được kết hợp vào trong một board thông qua hệ thống bus như mô tả trong Hình 1.

1.2 Sơ đồ khối và chức năng Kit SN8F5708_EVK

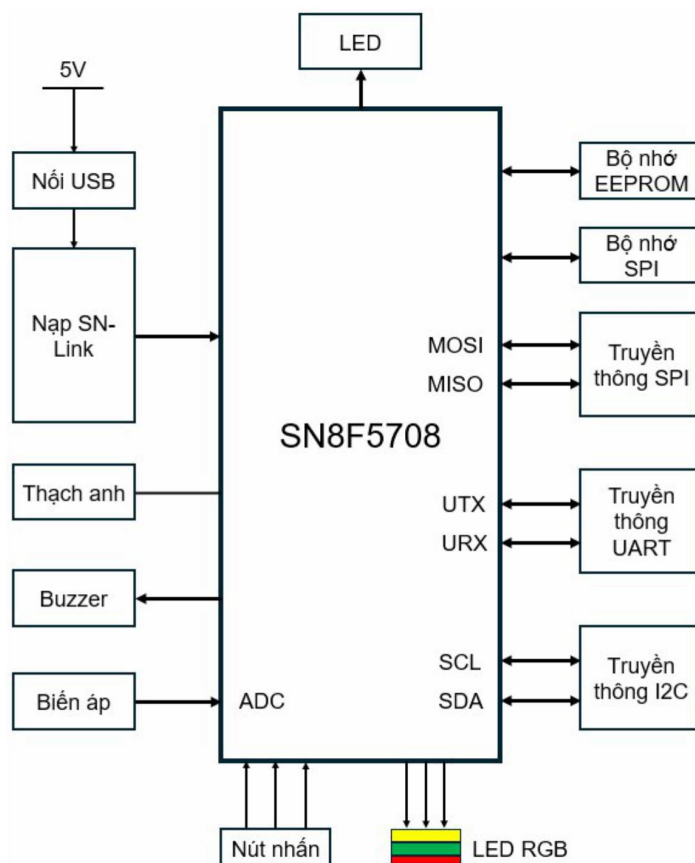
1.2.1 Kit SN8F5708_EVK

Kit được thiết kế và bố trí như mô tả trong Hình 2. Sinh viên có thể quan sát sơ đồ khối tối giản của Kit trên Hình 3.





Hình 2. Kit SN8F5708_EVK



Hình 3. Sơ đồ khối Kit SN8F5708_EVK

1.2.2 Chức năng

Kit SN8F5708_EVK thực hiện được nhiều chức năng cơ bản của 1 bộ Kit phát triển, đáp ứng nhu cầu làm quen với lập trình vi xử lý, ứng dụng để giải quyết vấn đề thực tế với tốc độ xử lý cao hơn hẳn vi điều khiển 8051 thông thường. Bảng 1 mô tả chức năng từng thành phần của Kit.

Bảng 1. Mô tả chức năng từng thành phần

STT	Kí hiệu	Mô tả
1	JP1	Đầu kết nối cable USB type-B
2	J8	Led báo hiệu chế độ Debug On: Chế độ Debug Off: Chế độ Normal
3	J5	Led báo hiệu kết nối cable USB On: Nguồn từ kết nối JP1 đã hoạt động Off: Không có nguồn từ JP1
4	SW1	Công tắc nguồn vào chip (MCU_VDD)
5	D3	Led báo hiệu nguồn chip
6	J2, J3	Chọn chế độ hoạt động Chế độ Normal: Chân SWAT và chân 8051_VDD được kết nối tới chân NORMAL Chế độ Debug: Chân SWAT và chân 8051_VDD được kết nối tới chân DEBUG
7	U2	MCU: SN8F5807FG
8	U3	SN-LINK V3 MCU
9	R51	Biến trở thay đổi điện áp đầu vào test ADC
10	SMG1	Led 7 thanh
11	SW2	Nút reset
12	PZ1	Buzzer
13	U4	Bộ đệm cho U3
14	U5	SPI Flash
15	U6	EEPROM
16	SW3~SW18	Ma trận nút nhấn
17	J4	Giao diện truyền thông UART
18	J7	Giao diện truyền thông SPI
19	J6	Giao diện truyền thông I2C
20	Y1	Thạch anh tạo dao động

2. Cài đặt phần mềm biên dịch và bộ công cụ dùng cho lập trình Kit SN8F5708_EVK

Học phần thí nghiệm, sinh viên sử dụng nền tảng lập trình KeilC_8051 và bộ công cụ (SDK) được cung cấp bởi GVHD. Sinh viên có thể dùng máy tính cá nhân hoặc máy tính trên phòng thí nghiệm (đã cài sẵn). Khuyến khích sinh viên dùng máy tính cá nhân thực hiện theo các bước được hướng dẫn dưới đây. Trong quá trình thực hiện, phát sinh vấn đề thì SV trao đổi trực tiếp với GVHD để cài đặt, lập trình và nạp code thành công trên máy tính từng sinh viên. Việc này giúp sinh viên thành thạo sử dụng các công cụ,

nền tảng KeilC trên chính máy tính của mình, ứng dụng trong các bài tập lớn và đồ án sau này (nếu cần).

2.1 SDK

2.1.1 SDK là gì

SDK (Software Development Kit) – bộ công cụ phát triển phần mềm là một gói các công cụ phát triển phần mềm. Chúng có thể dễ dàng tạo ra các ứng dụng nhờ có các trình biên dịch, trình gỡ lỗi và có thể bao gồm một software framework. Chúng thường dành riêng cho một nền tảng phần cứng và kết hợp hệ điều hành.

2.1.2 Tải SDK

GVHD cung cấp cho sinh viên đường dẫn đến thư mục chứa toàn bộ tài liệu dùng cho học phần thí nghiệm Vi xử lý. Sinh viên tham gia lớp MsTeams theo code sau để truy cập đường dẫn:

Khi tải về, sinh viên sẽ thu được các thư mục:

- Pack cho chip SN8F5708 (Pack)
- **11 dự án mẫu – nạp được luôn xuống Kit và xem đáp ứng** (Projects_Demo)
- Các thư viện và file liên kết sử dụng trong demo (Components)
- Datasheet
- Thư mục cài đặt KeilC_8051 (Keil C μ Vision 5 8051 C51V961)
- Hướng dẫn cài đặt KeilC_8051 và tài liệu thí nghiệm này.

2.2 Cài đặt phần mềm biên dịch

2.2.1 Cài đặt phần mềm KeilC_8051

Sinh viên cài đặt KeilC_8051 (sử dụng thư mục cài đặt Keil C μ Vision 5 8051 C51V961) **theo hướng dẫn gửi kèm bộ công cụ SDK – tham khảo vì khác phiên bản**, khi cài KeilC_8051 xong, chuyển sang bước sau.

2.2.2 Cài đặt pack cho chip SN8F5708 và tạo dự án mới

Pack cho chip được đặt cùng thư mục SDK mà GVHD cung cấp cho sinh viên. Sinh viên cài pack như cài 1 phần mềm bình thường. Lưu ý, sinh viên không thay đổi đường dẫn, pack sẽ mặc định chọn cài cùng KeilC mà sinh viên đã cài đặt trước đó (nếu muốn vị trí cài đặt khác với mặc định thì phải chọn từ bước cài KeilC).

Tạo dự án mới, lúc này chip SN8F5708 đã được add vào KeilC, lập trình như với các chip 8051 khác (tham khảo datasheet SN8F5708 để cấu hình thanh ghi đúng chức năng mong muốn).

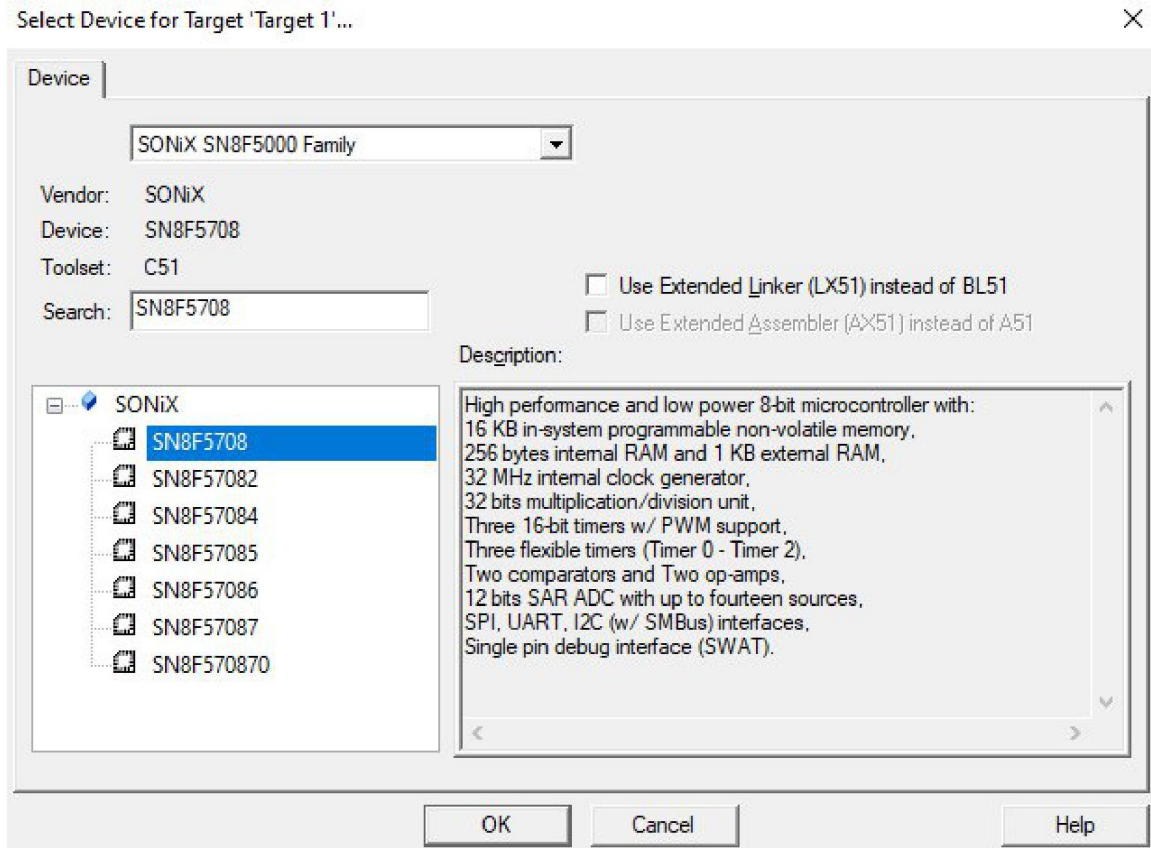
Lưu ý: Nếu phát triển ứng dụng tương tự như các bài trong nội dung thí nghiệm, sinh viên có thể copy dự án rồi chỉnh sửa file main, không cần add lại thư viện và tạo đường dẫn mới.

3. Biên dịch, nạp mã nguồn và kiểm tra đáp ứng của Kit với ví dụ nháy LED

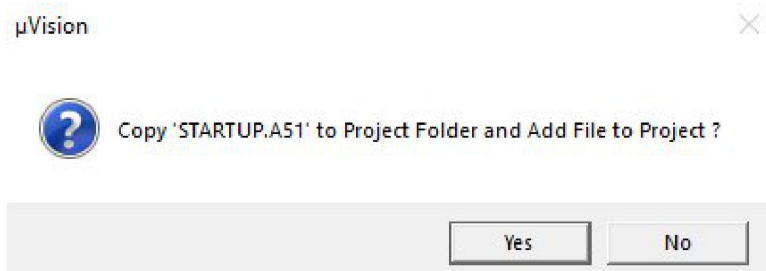
3.1 Tạo dự án và add các file mặc định cho Kit

3.1.1 Tạo dự án mới

- Project → New μ Vision Project → Chọn vị trí lưu dự án (tạo riêng 1 thư mục trong máy tính chứa tên dự án – AAA) và đặt tên dự án (AAA) → Cửa sổ Select Device for Target.



- Tại cửa sổ Select Device for Target, chọn dòng chip SONiX SN8F5000 Family → Chọn chip SN8F5708 → OK → Cửa sổ hỏi có tạo file STARTUP.A51 không → No.

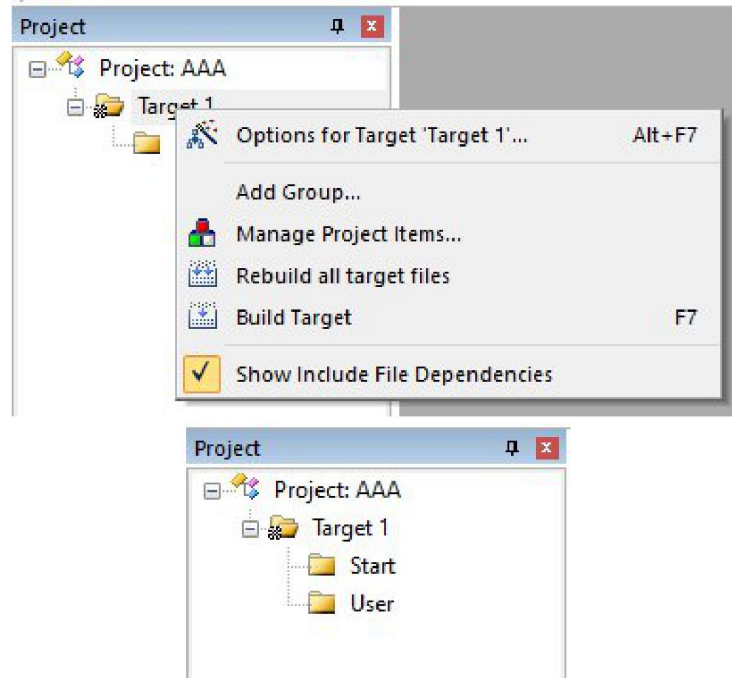


- Thư mục dự án vừa tạo trong máy tính (AAA), tạo thêm thư mục có tên Source để chứa các file thư viện, các file lập trình, các file định nghĩa và file liên kết của dự án. Copy toàn bộ các file trong thư mục Components (trong SDK) vào thư mục Source vừa tạo.

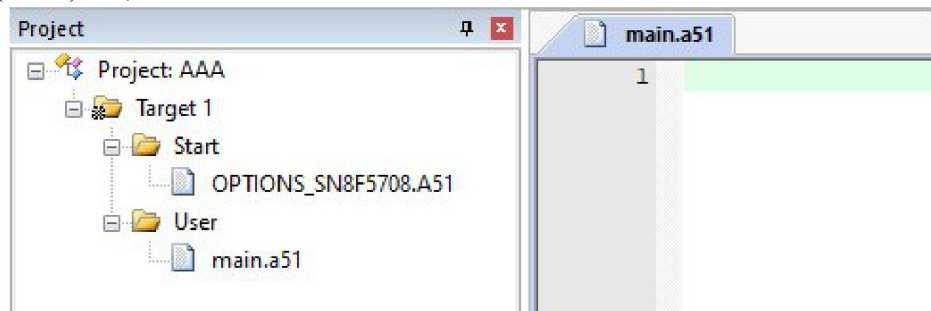
Name	Date modified	Type	Size
Listings	2/27/2025 10:30 AM	File folder	
Objects	2/27/2025 10:30 AM	File folder	
Source	2/27/2025 10:30 AM	File folder	

3.1.2 Add các file

- Tại cửa sổ Project, quan sát thấy cây dự án của mình. Kích chuột phải vào thư mục Target 1 → Add Group để tạo thư mục chứa file main và các file lập trình khác, sửa tên (User). Sửa tên thư mục Source Group 1 (mặc định) thành tên mong muốn (Start). Sửa tên bằng cách kích chuột trái 1 lần vào tên thư mục được chọn.



- Thư mục Start, add file định nghĩa Kit bằng cách kích chuột phải vào thư mục Start → Add Existing File → Chọn file OPTIONS_SN8F5708.a51 trong thư mục Source đã được tạo trong máy tính.
- Thư mục User, kích chuột phải → Add New Item → Chọn Asm File và đặt tên (main) hoặc add file có sẵn.



- Tương tự cho các file thư viện và file liên kết khác, sinh viên add các file muốn sử dụng trong thư mục Source nêu trên và làm tương tự các dự án demo được GVHD gửi trong SDK

3.2 Lập trình hàm main

- Lập trình theo mã nguồn sau vào file main.a51

```
$NOMOD51
```

```
STACK      SEGMENT IDATA
```

```
RSEG  STACK
```

```
DS 1
```

\$include (SN8F5708.H)

CSEG AT 00H

LJMP Reset

Reset:

MOV SP,#STACK - 1

;SP value is depend on user's RAM area define.

MOV WDTR,#5AH

CLR_RAM:

MOV R0,#0FFH

CLR A

CLR_RAM10:

MOV @R0,A

DJNZ R0,\$-1

=====

; Clear xdata(0X000-0X03FF)

=====

MOV R6,#00H

MOV R7,#04H

MOV DPTR,#00H

CLR_RAM20:

MOVX @DPTR,A

INC DPTR

DJNZ R6,CLR_RAM20

DJNZ R7,CLR_RAM20

CLR_RAM90:

MOV P0M,#0X18

Main:

MOV WDTR,#5AH

MOV R7,#4

ACALL DELAY_MS

XRL P0,#18H

Main90:

LJMP Main

RET

DELAY_MS:

MOV R5,#0FFH

MOV R4,#0FFH

DJNZ R4,\$

DJNZ R5,\$-2

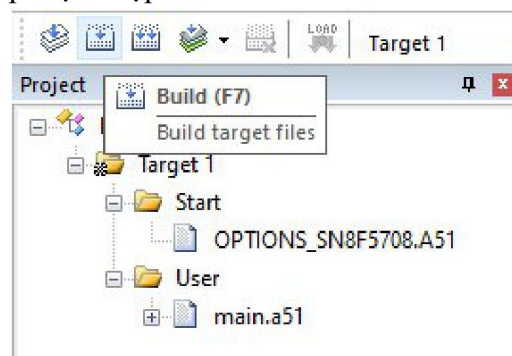
DJNZ R7,DELAY_MS

RET

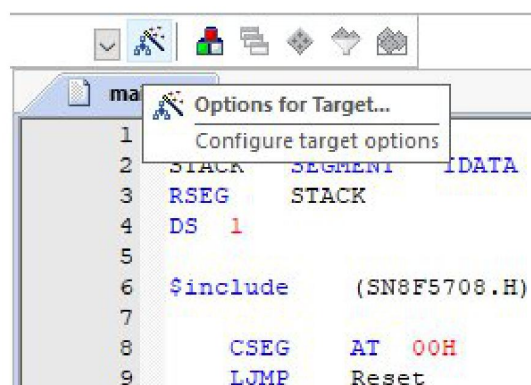
END

3.3 Biên dịch và Set up mạch nạp

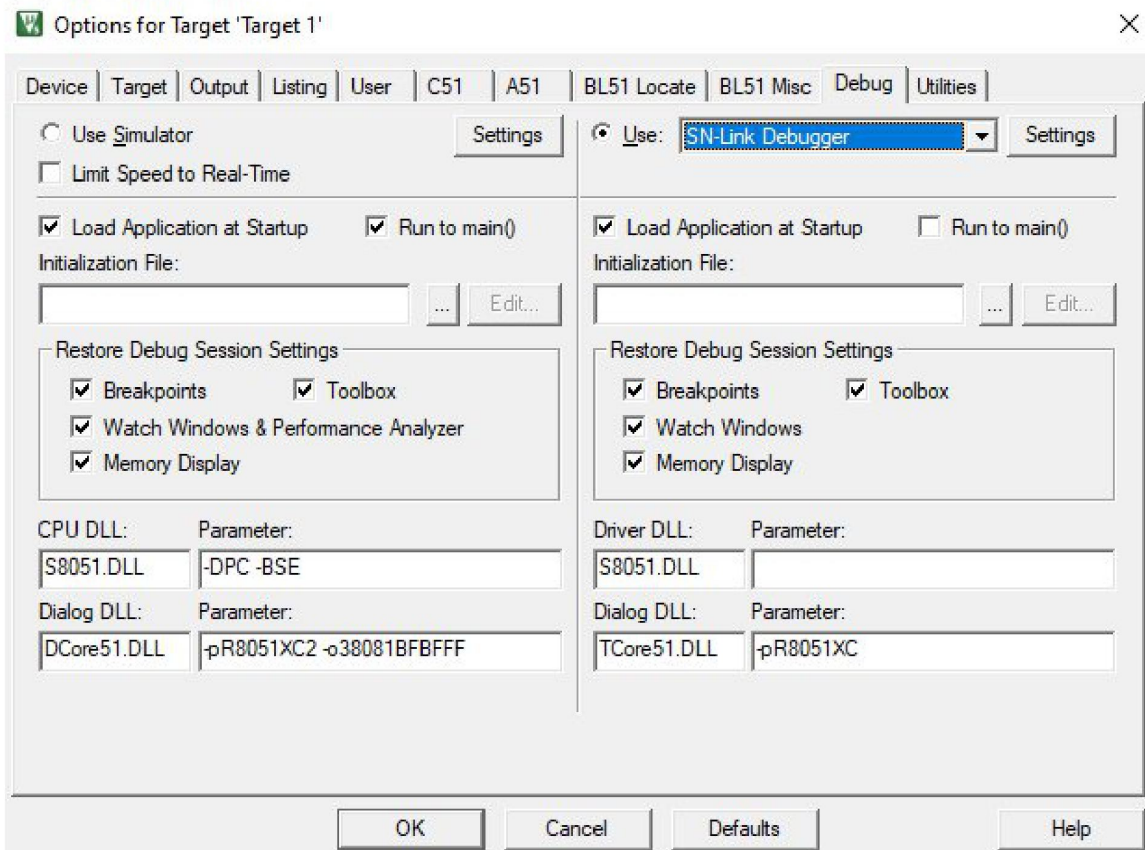
- Lưu dự án và biên dịch (F7). Lúc này ta thấy tại ô LOAD trên thanh công cụ bị ẩn đi, do chưa Set up mạch nạp.



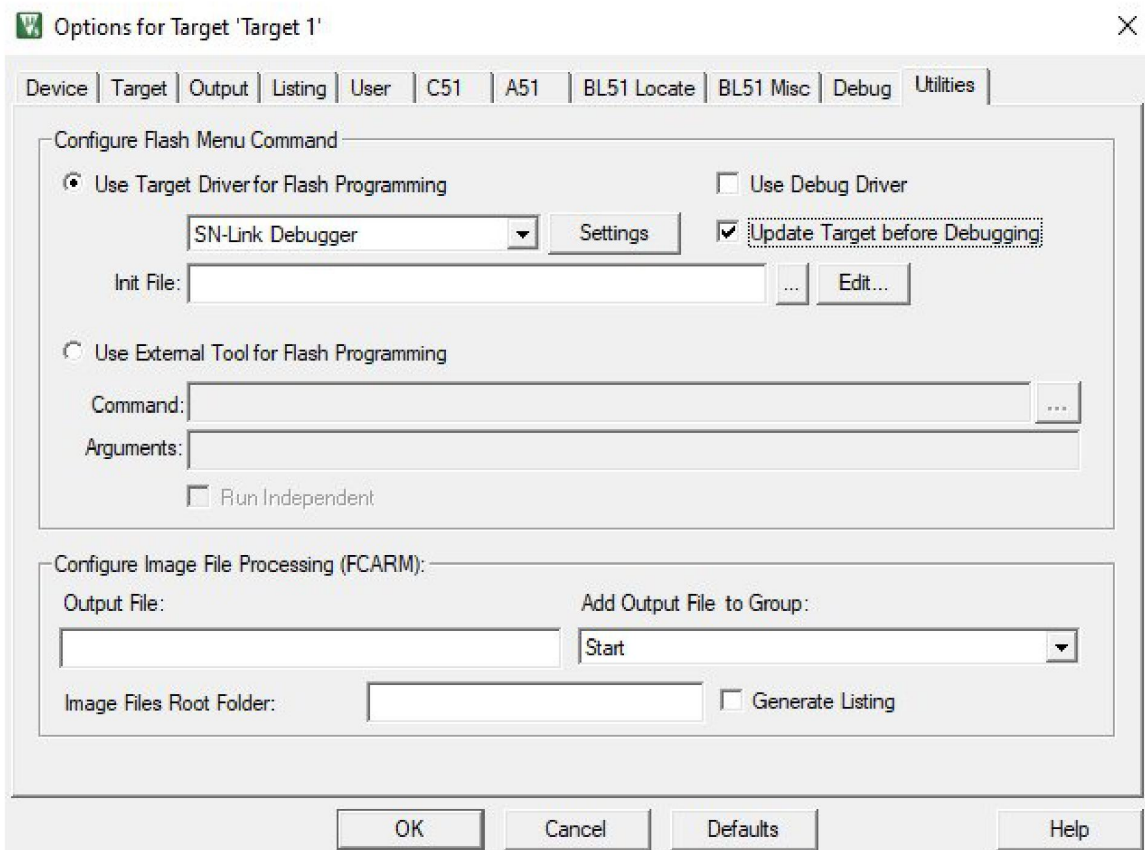
- Kích chọn ô Options for Target → cửa sổ Options for Target hiện ra để Set up mạch nạp.



- Tại ô Debug, tích chọn Use → Chọn mạch nạp SN-Link Debugger → Settings để quan sát trạng thái mạch nạp và Kit đã nhận đủ.



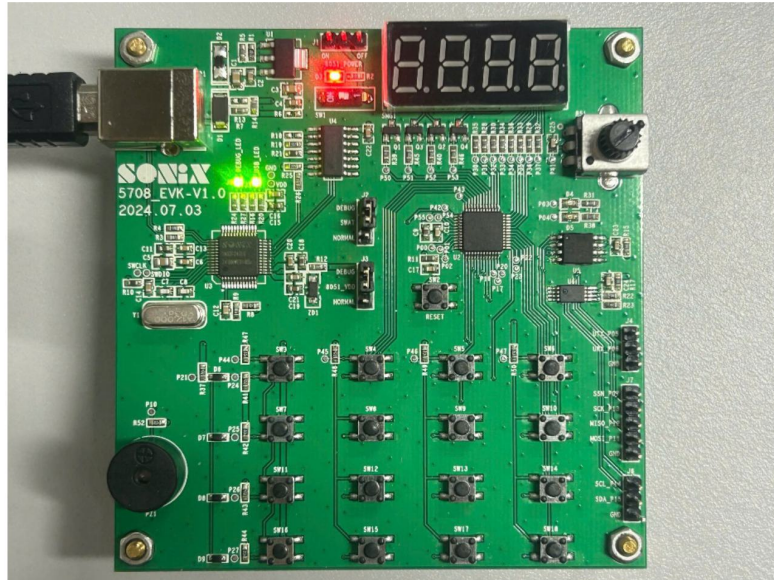
- Tại ô Utilities, tích chọn Use Target Driver for Flash Programming → Kéo cửa sổ cuộn chọn SN-Link Debugger → Update Target before Debugging → OK.



- Lúc này khi ấn LOAD (nạp code) thì trình biên dịch sẽ hiện ra thông báo lỗi “Error: failed to execute”. Vào lại Options for Target → Utilities → tích chọn Use Target Driver for Flash Programming → Ok → LOAD.

3.4 Nạp mã nguồn sử dụng cable USB Type-B

- Nối thông 2 chân Debug-Swat và Debug-8051_VDD (J2 và J3) sử dụng cầu nối đôi: đưa Kit vào chế độ Debug và nạp code.
- Tắt Switch nguồn (SW1) và bật lại đến khi 2 Led màu xanh lá (J8 và J5) sáng lên.



- Biên dịch và nạp code theo hướng dẫn ở trên.
- Sau khi nạp code xong, nối thông 2 chân Swat-Normal, 8051_VDD-Normal: đưa Kit vào chế độ thường và Run code.
- Tắt Switch nguồn (SW1) và bật lại để khởi động lại và run chương trình, quan sát đáp ứng trên Kit.

3.5 Nạp mã nguồn sử dụng mạch nạp SN-Link

Loading ...

3.6 Kết quả

Sinh viên sẽ quan sát thấy 2 Led D4 và D5 nhấp nháy theo thời gian Delay đã lập trình.

4. Công việc sinh viên cần thực hiện

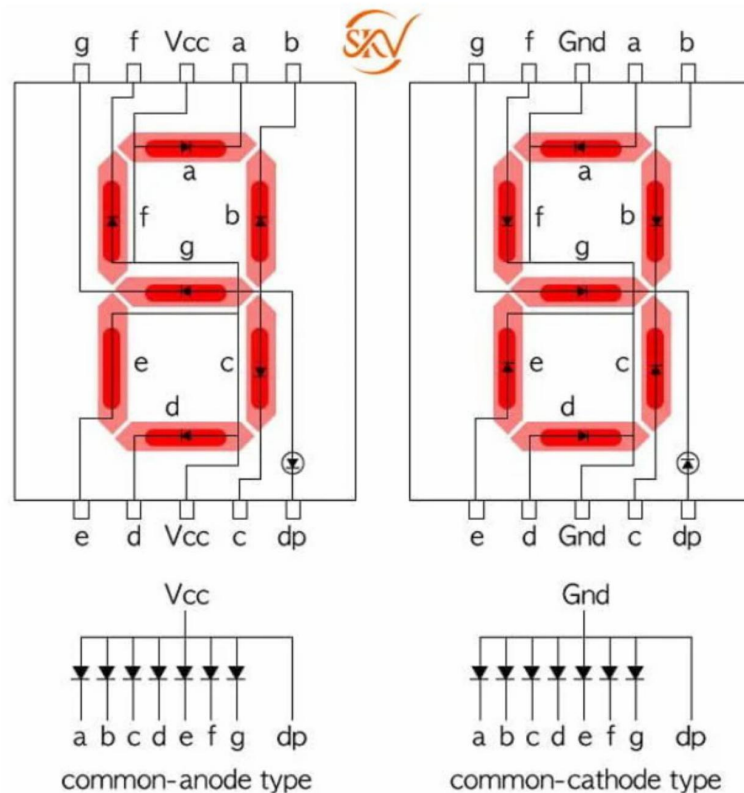
Sinh viên vận dụng 1 trong các dự án mẫu được gửi theo SDK (1 trong 6 bài dưới đây) để thực hiện theo đầu mục công việc từng buổi. Buổi 3, sinh viên nộp lại báo cáo tổng hợp nội dung thực hiện trong buổi 1 và buổi 2 (vào đầu giờ), sau đó triển khai nội dung công việc trong buổi 3 để GVHD đánh giá.

Bài 1. Hiển thị 4 chữ số Led 7 đoạn

1. Led 7 đoạn và nguyên lý hoạt động

1.1 Cấu tạo

LED 7 đoạn là một mô-đun hiển thị kỹ thuật số được thiết kế đặc biệt để hiển thị thông tin số. Nó bao gồm một số lượng thanh LED được sắp xếp theo hình dạng của các con số, tạo ra một màn hình dễ dàng nhìn thấy. Mỗi con số trong màn hình được tạo ra bằng cách bật hoặc tắt các thanh LED tương ứng. Cấu tạo LED 7 đoạn như được mô tả trong Hình 4.



Hình 4. Cấu tạo Led 7 đoạn

Nó được gọi là “7 đoạn” vì bao gồm 7 đoạn LED hàng đơn (segment) và một đoạn chấm thập phân (decimal point). Mỗi đoạn LED hàng đơn có thể hiển thị một trong 7 phần của các ký tự muốn hiển thị. Nếu hiển thị 2, nhiều ký tự hoặc số thập phân, ta kết hợp nhiều LED 7 đoạn sử dụng dấu chấm thập phân.

1.2 Nguyên lý hoạt động

Mỗi đoạn tương ứng với một thanh gạch ngang hoặc dọc của các chữ số. Mỗi đoạn LED đều có hai đầu điện cực, khi đầu dương được nối với dương nguồn và đầu âm nối với âm nguồn (GND) thì dòng điện sẽ chạy qua LED làm nó phát sáng (nguyên tắc phân cực thuận). Có 2 loại LED 7 đoạn: anode chung (các đầu dương nối chung vào dương nguồn – điều khiển bằng cách on/off đầu âm xuống âm nguồn) và cathode chung (các đầu âm nối chung vào âm nguồn – điều khiển bằng cách on/off đầu dương lên dương nguồn). Mô tả cách tạo ra chữ số hiển thị của LED 7 đoạn được mô tả trong Bảng 2.

Bảng 2. Bảng mô tả cách tạo ra chữ số a) anode chung; b) cathode chung

Số	Số nhị phân								HEX
	7	6	5	4	3	2	1	0	
	dp	g	f	e	d	c	b	a	
0	1	1	0	0	0	0	0	0	C0
1	1	1	1	1	1	0	0	1	F9
2	1	0	1	0	0	1	0	0	A4
3	1	0	1	1	0	0	0	0	B0
4	1	0	0	1	1	0	0	1	99
5	1	0	0	1	0	0	1	0	92
6	1	0	0	0	0	0	1	0	82
7	1	1	1	1	1	0	0	0	8F
8	1	0	0	0	0	0	0	0	80
9	1	0	0	1	0	0	0	0	90
A	1	0	0	0	1	0	0	0	88
B	1	0	0	0	0	0	1	1	83
C	1	1	0	0	0	1	1	0	C6
D	1	0	1	0	0	0	0	1	A1
E	1	0	0	0	0	1	1	0	86
F	1	0	0	0	1	1	1	0	8E

a)

Số	Số nhị phân								HEX
	7	6	5	4	3	2	1	0	
	dp	g	f	e	d	c	b	a	
0	0	0	1	1	1	1	1	1	0x3F
1	0	0	0	0	0	1	1	0	0x06
2	0	1	0	1	1	0	1	1	0x5B
3	0	1	0	0	0	0	0	0	0x40
4	0	1	1	0	0	1	1	0	0x66
5	0	1	1	0	1	1	0	1	0x6D
6	0	1	1	1	1	1	0	1	0x7D
7	0	0	0	0	0	1	1	1	0x07
8	0	1	1	1	1	1	1	1	0x7F
9	0	1	1	0	1	1	1	1	0x6F
A	0	1	1	1	0	1	1	1	0x77
B	0	1	1	1	1	1	0	0	0x7C
C	0	0	1	1	1	0	0	1	0x39
D	0	1	0	1	1	1	1	0	0x5E
E	0	1	1	1	1	0	0	1	0x79
F	0	1	1	1	0	0	0	1	0x71

b)

- 2. Làm quen với Kit thí nghiệm, vẽ lưu đồ thuật toán và sơ đồ kết nối chân – Buổi 1**
- 3. Tạo dự án, add các thư viện và lập trình cho Kit (sinh viên lập trình và giải thích code của mình bao gồm file thư viện) – Buổi 2**
- 4. Nhúng code xuống Kit, kiểm tra đáp ứng, đánh giá và kết thúc thí nghiệm – Buổi 3**

Bài 2. Điều khiển Led 7 thanh bằng phím nhấn

- 1. Làm quen với Kit thí nghiệm, vẽ lưu đồ thuật toán và sơ đồ kết nối chân – Buổi 1**
- 2. Tạo dự án, add các thư viện và lập trình cho Kit (sinh viên lập trình và giải thích code của mình bao gồm file thư viện) – Buổi 2**
- 3. Nhúng code xuống Kit, kiểm tra đáp ứng và kết thúc thí nghiệm – Buổi 3**

Bài 3. Hiện thị giá trị mã khóa được nhận

- 1. Làm quen với Kit thí nghiệm, vẽ lưu đồ thuật toán và sơ đồ kết nối chân – Buổi 1**
- 2. Tạo dự án, add các thư viện và lập trình cho Kit (sinh viên lập trình và giải thích code của mình bao gồm file thư viện) – Buổi 2**
- 3. Nhúng code xuống Kit, kiểm tra đáp ứng và kết thúc thí nghiệm – Buổi 3**

Bài 4. Đo giá trị điện áp trên biến trở R51

1. ADC và cơ chế đọc điện áp

1.1 Bộ chuyển đổi ADC là gì

1.1.1 Định nghĩa

ADC (Analog-to-Digital Converter) là bộ chuyển đổi tín hiệu tương tự (Analog) thành tín hiệu số (Digital). ADC giúp các thiết bị số (như vi điều khiển, máy tính) có thể đọc và xử lý tín hiệu từ thế giới thực (âm thanh, nhiệt độ, ánh sáng, áp suất, ...) thông qua cảm biến (chuyển đổi tín hiệu phi điện thành tín hiệu điện) và nhiều ứng dụng khác. Tín hiệu tương tự được lấy mẫu sang tín hiệu số với số lượng lớn mẫu, mẫu càng nhiều thì tái hiện lại tín hiệu tương tự càng chính xác.

1.1.2 Số bit và điện áp tham chiếu của bộ ADC

- Số bit

ADC có N-bit thì tín hiệu đầu vào sẽ được chia thành 2^N mức.

Ví dụ: ADC 16-bit chia thành $2^{16} = 65536$ mức.

- Điện áp tham chiếu (V_{ref})

Điện áp tham chiếu (V_{ref}) là mức **điện áp tối đa** mà ADC có thể đo được (vượt quá điện áp tham chiếu, bộ ADC sẽ không đọc được, thậm chí đánh hỏng bộ ADC và VĐK nên cần mạch bảo vệ cho bộ ADC khi thiết kế mạch). Giá trị điện áp đầu vào được chia thành nhiều mức theo điện áp tham chiếu.

Công thức tính độ phân giải điện áp của ADC = $\frac{V_{ref}}{2^N}$ (để hiểu đơn giản thì độ phân giải điện áp là khoảng cách giữa 2 mẫu gần nhau nhất khi lấy mẫu).

1.2 Nguyên lý đọc giá trị điện áp

1.2.1 Lấy mẫu

Tín hiệu điện áp sẽ được VĐK rời rạc hóa, ví dụ với bộ ADC 8-bit, $V_{ref} = 5V$ thì điện áp đầu vào sẽ trải dài rời rạc cách đều từ 0V (tại mẫu số 0) đến 5V (tại mẫu số $2^8 - 1 = 255$).

1.2.2 Tần số lấy mẫu

- Là số lần ADC có thể đo và chuyển đổi tín hiệu trong một giây, tính bằng đơn vị mẫu/giây hoặc Hz. Ví dụ: Một ADC có tốc độ 1kSPS (1000 mẫu/giây) nghĩa là nó có thể đo 1000 lần trong 1 giây.
- Tốc độ lấy mẫu càng cao thì khả năng theo dõi tín hiệu nhanh càng tốt và sự chính xác càng cao.

1.2.3 Mối tương quan

Giá trị đọc được từ bộ ADC sẽ tương quan với giá trị điện áp đầu vào theo công thức:

$$ADC_{value} = \frac{V_{in}}{V_{ref}} * 2^N - 1$$

Ở đây, ADC_{value} là giá trị số nhận được từ ADC.

V_{in} là điện áp đầu vào.

V_{ref} là điện áp tham chiếu của ADC.

N là độ phân giải của ADC (8-bit, 12-bit, ...)

- 2. Làm quen với Kit thí nghiệm, vẽ lưu đồ thuật toán và sơ đồ kết nối chân – Buổi 1**
- 3. Tạo dự án, add các thư viện và lập trình cho Kit (sinh viên lập trình và giải thích code của mình bao gồm file thư viện) – Buổi 2**
- 4. Nhúng code xuống Kit, kiểm tra đáp ứng và kết thúc thí nghiệm – Buổi 3**

Bài 5. Điều chỉnh cao độ - tần số của Buzzer PZ1

1. Buzzer và nguyên lý điều khiển

1.1 Buzzer là gì

Buzzer có nghĩa là thành phần điện tử tạo ra âm thanh thông qua việc truyền tín hiệu điện. Chức năng chính của nó là cung cấp một cảnh báo hoặc thông báo có thể nghe được và thường hoạt động trong phạm vi điện áp từ 5V đến 12V. Có nhiều loại mô-đun khác nhau trong các cơ chế tạo âm thanh, trong đó buzzer hoạt động dựa trên hiện tượng cảm ứng điện từ. Buzzer gồm một cuộn dây điện từ và một màng rung kim loại (diaphragm).

1.2 Nguyên lý hoạt động

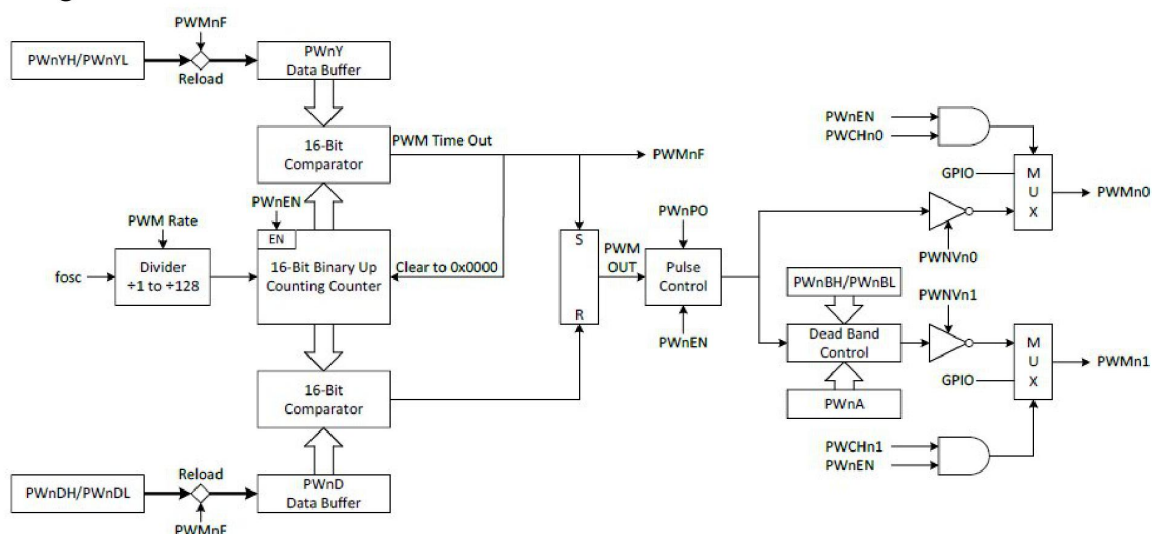
- Khi có dòng điện chạy qua cuộn dây, nó tạo ra từ trường.
- Từ trường hút màng rung kim loại về phía cuộn dây.
- Khi ngắt dòng điện, màng rung trở lại vị trí ban đầu nhờ lực đàn hồi.
- Chu kỳ này lặp đi lặp lại theo tần số dòng điện, tạo ra âm thanh.

Khác với buzzer chủ động có mạch dao động bên trong nên chỉ cần cấp nguồn là mạch phát một âm thanh dài, còn còi buzzer thụ động không có bộ dao động bên trong nên khi cấp một tần số, còi sẽ phát âm thanh thanh tùy theo tần số và thời gian.

Buzzer thụ động cần một tín hiệu dao động từ vi điều khiển để tạo ra âm thanh. Vi điều khiển phải cung cấp một tín hiệu xung PWM (Pulse Width Modulation) với tần số phù hợp để buzzer có thể phát ra âm thanh.

1.3 Điều khiển sử dụng VĐK SN8F5708

PWM (Pulse Width Modulation) là kỹ thuật điều chế độ rộng xung, giúp tạo tín hiệu xung vuông có tần số cố định nhưng độ rộng xung thay đổi. Ý tưởng cơ bản đằng sau việc triển khai PWM là sử dụng Timers và chuyển đổi port pin high/low theo các khoảng thời gian xác định tạo ra các tần số tín hiệu khác nhau. Sơ đồ điều khiển được thể hiện trong Hình 5.



Hình 5. Sơ đồ điều khiển bộ PWM của VĐK SN8F5708

Sinh viên tham khảo các đoạn code mẫu trong datasheet hoặc ví dụ demo

- 2. Làm quen với Kit thí nghiệm, vẽ lưu đồ thuật toán và sơ đồ kết nối chân – Buổi 1**
- 3. Tạo dự án, add các thư viện và lập trình cho Kit (sinh viên lập trình và giải thích code của mình bao gồm file thư viện) – Buổi 2**
- 4. Nhúng code xuống Kit, kiểm tra đáp ứng và kết thúc thí nghiệm – Buổi 3**

Bài 6. Truyền thông UART

1. Nguyên lý truyền thông UART

1.1 Khái niệm UART và USART

- UART (Universal Asynchronous Receiver-Transmitter – Bộ truyền nhận dữ liệu không đồng bộ) là một giao thức truyền thông phần cứng dùng giao tiếp nối tiếp không đồng bộ và có thể cấu hình được tốc độ truyền.
- USART (Universal Synchronous Receiver-Transmitter – Bộ truyền nhận dữ liệu đồng bộ) có thêm khả năng hoạt động trên BUS (tồn tại master/slave), ngoài ra nó cũng có chế độ hoạt động “đồng bộ” với nguồn xung giữ nhịp bên ngoài (do chip khác cấp).

Trong bài này, sinh viên sử dụng giao thức truyền thông UART. Giao thức UART là một giao thức đơn giản và phổ biến, bao gồm hai đường truyền dữ liệu độc lập là TX (truyền) và RX (nhận). Dữ liệu được truyền và nhận qua các đường truyền này dưới dạng các khung dữ liệu (data frame) có cấu trúc chuẩn, với một bit bắt đầu (start bit), một số bit dữ liệu (data bits), một bit kiểm tra chẵn lẻ (parity bit) và một hoặc nhiều bit dừng (stop bit).

Thông thường, tốc độ truyền của UART được đặt ở một số chuẩn, chẳng hạn như 9600, 19200, 38400, 57600, 115200 baud và các tốc độ khác. Tốc độ truyền này định nghĩa số lượng bit được truyền qua mỗi giây. Các tốc độ truyền khác nhau thường được sử dụng tùy thuộc vào ứng dụng và hệ thống sử dụng.

1.2 Truyền thông UART sử dụng VĐK SN8F5708

1.2.1 Cấu hình chân UTX và URX

Các chân UART UTX và URX được cấu hình cùng với chức năng GPIO.

- Chế độ đồng bộ: các chân UTX/URX phải được thiết lập mức cao bằng phần mềm.
- Chế độ bất đồng bộ (UART 8-bit/9-bit): chân UTX phải được đặt mức cao, còn chân URX phải được đặt làm đầu vào mức cao bằng phần mềm.

Khi UART bị vô hiệu hóa, các chân UART sẽ trở về trạng thái GPIO trước đó. Các chân UTX/URX hỗ trợ cấu trúc open-drain. Tùy chọn này được kiểm soát bởi bit PnOC. Khi chế độ open-drain được kích hoạt, chân UTX/URX phải được đặt mức cao (chế độ IO sẽ bị bỏ qua) và cần có điện trở pull-up bên ngoài.

- Khi PnOC = 0, vô hiệu hóa chế độ open-drain của UTX/URX.
- Khi PnOC = 1, kích hoạt chế độ open-drain của UTX/URX.

1.2.2 Ngắt UART

UART hỗ trợ chức năng ngắt. ES0 là bit điều khiển chức năng ngắt truyền UART0.

- Khi ES0 = 0, vô hiệu hóa ngắt truyền/nhận.
- Khi ES0 = 1, kích hoạt ngắt truyền/nhận.

UART chia sẻ vector ngắt 0x0023 cho cả bộ thu và phát. Khi ngắt được kích hoạt, bộ đếm chương trình (PC) sẽ trở đến vector ngắt để thực hiện trình phục vụ ngắt UART. TI0/RI0 là cờ yêu cầu ngắt của UART0 và cũng là chỉ báo trạng thái hoạt động của UART khi ngắt bị vô hiệu hóa. TI0 và RI0 phải được xóa bằng phần mềm.

Sinh viên tham khảo các đoạn code mẫu trong datasheet hoặc ví dụ demo

- 2. Làm quen với Kit thí nghiệm, vẽ lưu đồ thuật toán và sơ đồ kết nối chân – Buổi 1**
- 3. Tạo dự án, add các thư viện và lập trình cho Kit (sinh viên lập trình và giải thích code của mình bao gồm file thư viện) – Buổi 2**
- 4. Nhúng code xuống Kit, kiểm tra đáp ứng và kết thúc thí nghiệm – Buổi 3**